

Interpretive Coupling

Semantic Slack, Specification Completion, and Consequential Language in Prompt-Mediated Writing

Watson Hartsoe

Working paper — August 17, 2026

Abstract

Natural-language prompting is often described by analogy: it is “like programming,” “between code and poetry,” or a new interface for specifying what a computer should do. These descriptions identify real features but obscure a more consequential transformation. Contemporary generative systems can couple highly underspecified symbolic expressions to computational consequences while using learned priors and situated context to complete parts of the specification during realization. I call this configuration *interpretive coupling*. It has three components: semantic slack, the degree to which consequential aspects of a result remain unstated in an expression; consequence coupling, the degree to which an interpretation can alter reachable system states; and specification completion, the process by which candidate realizations make unstated commitments concrete and thereby elicit further human judgment. This account shifts analysis from the prompt string to a recursive prompt-event in which specification, realization, judgment, and revision can be distributed across actors and across time. The argument also limits claims of novelty. Speech acts, situated action, tacit knowledge, declarative programming, and distributed authorship all precede large language models. What is distinctive is not that ambiguous language suddenly acquired effects, but that learned computational interpreters make it cheap, rapid, and repeatable to bind incompletely formalized descriptions to generative action. The paper develops this claim, proposes discriminating experiments, and argues that provenance should preserve decision structure without pretending to settle authorship.

Keywords: prompting; natural-language programming; specification; interpretation; human–AI interaction; authorship; tacit knowledge; generative systems

1. The wrong border

Prompting is frequently theorized through a border metaphor. Formal programs sit on one side, literary language on the other, and prompts occupy an unstable region between them. The metaphor is attractive because it captures a felt difference: a conventional program is expected to constrain admissible interpretation, while a prompt can succeed despite ordinary-language ambiguity, metaphor, ellipsis, and context dependence. Early work explicitly described interaction with large language models as “prompt programming” and emphasized the usefulness of natural language, narratives, and cultural anchors for controlling model behavior (Reynolds and McDonell 2021). A

large survey literature has meanwhile formalized a narrower object, prompt-based learning, in which an input is transformed into a textual prompt with slots that a language model probabilistically completes (Liu et al. 2023).

The phrase “between code and poetry,” however, makes two independent properties look like one continuum. The first is how much of an intended realization is explicitly determined by an expression. The second is how tightly interpretation of that expression is coupled to a system capable of changing state. Poetry can leave enormous interpretive latitude while remaining weakly coupled to immediate computational action. A formal program can couple symbols tightly to state transitions while deliberately constraining interpretation. Prompting can combine the two: an expression can leave consequential details unresolved and still be accepted as sufficient input to machinery that produces code, images, actions, or worlds.

This paper calls the first property *semantic slack* and the second *consequence coupling*. These names are provisional. They are analytical handles, not established metrics, and neither property belongs uniquely to prompting. Natural-language contracts, commands, laws, vows, and institutional classifications can be both interpretively open and consequential. Declarative and constraint-oriented programs can leave implementation choices unresolved. The proposal is therefore not a taxonomy in which “prompt” names one quadrant. It is a claim that generative systems make one configuration newly accessible at scale: high interpretive openness can coexist with strong computational consequence because a learned interpreter supplies structure not explicitly encoded in the user expression.

That reframing replaces a genre question—what kind of writing is a prompt?—with a mechanism question: *what machinery allows partially specified language to reorganize what can happen next?*

2. From prompt string to prompt-event

A visible prompt is often a poor unit of analysis. Consider the expression “Again.” As an autonomous text it specifies almost nothing. Within an ongoing interaction it may be entirely sufficient. It can refer to a previous generation, preserve the current task, inherit a style, repeat an operation, or request another sample from a distribution. The expression works because something outside the expression has already been made addressable.

This is familiar from pragmatics and from studies of situated action. Geertz’s account of thick description famously turns on the fact that a movement cannot be identified adequately without its interpretive context (Geertz 1973). Suchman similarly challenged plans-as-scripts models of action by emphasizing that action is situated and that plans operate as resources within unfolding circumstances rather than exhaustive determinants of behavior (Suchman 1987). Neither account concerns modern prompting, but both weaken the idea that the meaning of an operative expression can be read off its visible syntax alone.

For prompt-mediated systems the additional issue is *computational addressability*. Conversation history, selected image regions, files, tool results, interface state, persistent memory, and previously generated objects can become referents against which a short expression is resolved. A useful unit of

analysis is therefore a *prompt-event*: an expression together with the available context, interpreter, action repertoire, resulting candidate, and subsequent human judgment.

Formally, we might write the interpretation of an expression E as a function of more than E itself:

$$I = f(E, C, M, S, X),$$

where C is addressable context, M is the learned model, S is system-level constraint, and X includes examples or tool affordances. This is not intended as a model of neural computation. It is an accounting device that prevents the analyst from treating the prompt string as the entire specification.

The consequence is methodological. Two identical strings can be different operative objects if their contexts differ. Conversely, different strings can converge on similar behavior because a learned model supplies comparable interpretations. A theory of prompting that studies only lexical form will therefore miss the distribution of semantic work across expression, context, and interpreter.

3. Semantic slack and consequence coupling

Semantic slack is the portion of consequential realization that an expression leaves unresolved. The term should not be reduced to lexical ambiguity. At least six forms need to be separated: unspecified values, unspecified procedures, context dependence, tolerance for variation, reliance on learned priors, and underdetermined evaluative criteria. “Make the room warmer” may leave open whether warmer means temperature, color, social atmosphere, or visual grading. “Write a detective story in which every apparently supernatural event receives a mundane explanation” may sharply constrain a structural invariant while leaving nearly every sentence and event unspecified.

Consequence coupling concerns the relation between interpretation and reachable system states. If a representation is supplied to a system and its interpretation changes available actions, constraints, probabilities, artifacts, or world state, then the representation participates in a consequential transition. A simple state-based notation is useful. Let T_0 denote a system’s transition relation before interpreting representation r , and T_r the relation after interpretation. A representation is operationally consequential when $T_r \neq T_0$: some state becomes newly reachable, unreachable, or differently probable.

This formulation deliberately avoids saying that “description becomes operation.” Description does not acquire magical execution semantics by itself. Austin’s analysis of performative utterance already showed that words can participate in doing things under the appropriate conventional conditions (Austin 1962). The relevant question is what mediating machinery and institutions connect an expression to consequence. In prompt-mediated systems the chain may include model inference, tool selection, permissions, external APIs, rendering systems, or simulated worlds. The same sentence can be inert prose in one setting and a state-changing intervention in another.

The resulting conceptual field is two-dimensional rather than linear:

	Lower consequence coupling	Higher consequence coupling
Lower semantic slack	literal notation, fixed description	formal programs, tightly specified commands
Higher semantic slack	poetry, metaphor, open description	generative prompting, situated natural-language control

The table is not a classification of genres. Every cell contains counterexamples. Its use is diagnostic: it asks which property changed and whether a supposed novelty survives comparison with older forms of consequential language.

4. The interpreter does more than execute

The strongest reason to move beyond the code/poetry border is that generative models can participate in specification itself. A conventional story about programming runs from specification to execution: the programmer states what should happen, and the machine follows the resulting formal structure. Prompting often has a different temporal architecture.

A user writes “Make the house remember everyone who lived there.” The phrase does not state what remembering looks like, which events should be stored, whether memory is visible to inhabitants, how it changes architecture, or how old memories decay. A generative model must make commitments to realize any concrete version. Those commitments externalize one interpretation of the instruction. The user can then respond: “No—not records. I want the house to behave differently because of them.” A requirement appears only after an attempted realization has made one previously tacit interpretation visible.

Call this *specification completion*. The model does not merely execute a finished description. It supplies candidate commitments while realizing it. Human judgment then accepts, rejects, or transforms those commitments, and the resulting judgment can re-enter as a new symbolic intervention. A minimal representation is:

$$S_t \rightarrow C_t \rightarrow R_t \rightarrow J_t \rightarrow S_{t+1},$$

where S_t is the explicit specification, C_t an interpretation-dependent completion, R_t a realization, and J_t human judgment. Importantly, S_{t+1} can contain criteria absent from S_t .

This mechanism needs to be distinguished from a rival account already present in prompt research. Reynolds and McDonell suggest that few-shot examples can sometimes be better understood as locating an already learned task than as learning a new task from the prompt (Reynolds and McDonell 2021). On that account, missing structure does not arise only because an interpreter completes a novel specification during inference; some structure was learned during pretraining and the prompt functions partly as an index or locator. “Specification completion” and “task

location” are therefore competing mechanisms, not synonyms. A serious theory of prompting needs experiments capable of separating them.

One useful test is to hold the target behavior fixed while varying the expression from minimal task labels to examples to detailed constraints. If successful behavior appears under sparse labels and remains stable across rephrasings, task location becomes plausible. If behavior changes monotonically as omitted constraints are added, specification completion becomes more plausible. Many real interactions will involve both.

5. Failure as negative specification

Specification completion becomes particularly visible in failure. A generated candidate can reveal that a user possessed a discrimination without possessing a prior proposition that expressed it. The user says “No.” Only afterward do they discover what was wrong.

Polanyi’s well-known formulation that humans can know more than they can tell provides a useful boundary condition (Polanyi 1966). It does not prove that aesthetic judgment is ineffable, nor does it justify every unexplained preference. It does establish that explicit verbal statement need not exhaust knowledgeable discrimination. Schön’s account of reflection-in-action similarly emphasizes how practitioners reshape problems in the course of acting rather than simply applying complete prior rules (Schön 1983).

Prompt-mediated interaction turns this familiar feature of practice into an observable feedback loop. A candidate is produced; the human recognizes a violation; a rejection, edit, ranking, gesture, or example externalizes some portion of that discrimination; the system uses the signal to generate another candidate. I call this *negative specification*: the progressive narrowing of a possibility space through recognized violations before the relevant positive criterion is fully articulated.

The phrase requires caution. A bare “no” does not identify which property caused failure. The model may infer the wrong boundary, and the human may rationalize a preference after the fact. The important research problem is therefore not whether tacit knowledge exists but how tacit discrimination becomes *machine-addressable feedback*. Different interfaces perform that conversion differently. Binary rejection, pairwise ranking, direct manipulation, textual critique, exemplar selection, and hand editing all expose different projections of the evaluator’s judgment.

This suggests an empirical program. Ask participants to create an artifact they report being able to recognize but not fully specify. Compare feedback modalities while measuring convergence, consistency, the amount of explicit criteria produced, and the degree to which later explanations predict future judgments. The result would help determine whether “negative specification” is a genuine mechanism or a flattering name for post-hoc correction.

6. Writing as a distributed operation

Once specification can be completed through realization and judgment, “writing” becomes difficult to treat as one indivisible operation. At minimum, a prompt-mediated process can separate problem

formulation, constraint setting, immediate inscription, candidate generation, recognition, rejection, selection, revision, arrangement, and responsibility. These operations can be distributed across people and systems and can recur over time.

This does not by itself imply that authorship has disappeared. It does show why the binary question “Did the human write it?” can be too coarse for analysis. A complete production process is better represented as a graph of operations and dependencies than as a single causal arrow from author to text. The author label then behaves partly as a compression of that graph.

But “compression” is not a sufficient theory of authorship. Foucault’s author-function offers a stronger opposition. In his account, an author is not simply the natural name attached to an originating individual; the author-function varies across kinds of discourse and is constructed through historically specific operations of appropriation, classification, comparison, continuity, and exclusion (Foucault 1977). The key implication for generative work is that a production graph and an author-function answer different questions. The graph asks what operations produced the artifact. The author-function asks which operations an institution or discourse treats as author-making.

A dissertation committee, a copyright office, a film credit system, and a scientific contributor taxonomy can encounter the same causal production history and construct different attribution regimes. Generative media therefore do not merely distribute authorship. They expose mismatches among existing author-functions by separating activities that older institutions often treated as bundled.

This reframes provenance. A prompt receipt cannot prove intention, understanding, or authorship simply by logging every interaction. Exhaustive logs can be less interpretable than curated records. Their stronger function is to preserve distinctions that a coarse attribution label would erase: which alternatives appeared, what was rejected, where a criterion changed, what source was introduced, what the model supplied unexpectedly, and which decision reorganized the downstream possibility space.

7. Decision leverage and the limits of provenance

A useful provenance record should therefore preserve decision structure rather than count textual labor. A two-word correction can have greater intellectual consequence than a thousand-word prompt if it removes an entire branch of possible argument. Conversely, an automated action can have enormous causal influence without constituting intellectual contribution.

This motivates a tentative concept of *decision leverage*: the degree to which an intervention changes downstream reachable possibilities. The concept is intentionally narrower than authorship. It does not measure originality, effort, moral responsibility, or understanding. It asks only where the production trajectory materially changed.

In principle, decision leverage can be probed counterfactually. Remove an intervention, replace it with a neutral alternative, and regenerate the downstream process. Large structural divergence suggests high causal leverage. Such tests would be noisy for stochastic systems and impossible for many historical artifacts, but even approximate counterfactuals are more informative than prompt

counts.

The deeper point is that provenance is not self-interpreting. A complete transcript still requires a theory of relevance. Which branches matter? Which edits were conceptual rather than cosmetic? Which generated novelty was adopted rather than merely tolerated? A provenance system therefore becomes an authored representation of production, not a transparent window onto it.

8. What is actually new?

The argument so far deliberately destroys several easy novelty claims. Context-dependent meaning is older than language models. Consequential language is older than computation. Tacit judgment is older than prompting. Distributed authorship is older than generative AI. Declarative specifications already separate desired conditions from execution paths. Prototyping already allows requirements to emerge through realization.

What, then, changes?

The best current answer is a change in the cost and scale of *interpretive coupling*. Large generative models can accept expressions whose consequential detail remains incompletely formalized, combine them with learned priors and addressable context, produce rich candidate realizations rapidly, and accept the user’s subsequent judgment as another input to the same loop. This makes it practical to defer portions of formalization until after a candidate exists.

The novelty claim is therefore architectural rather than metaphysical. No new essence called “prompt language” is required. Instead, several older capacities are assembled into a new control regime:

1. **Learned semantic burden.** The interpreter supplies large bodies of unstated structure acquired during training.
2. **Situated addressability.** Current objects, histories, selections, and tools can be referenced with sparse expressions.
3. **Generative realization.** One partial interpretation can be externalized as a rich candidate rather than merely classified.
4. **Recursive judgment.** Human recognition, rejection, and revision can return immediately as new symbolic constraint.
5. **Low-cost iteration.** The loop can be repeated quickly enough that specification itself can emerge through interaction.

Together these features produce interpretive coupling: incomplete descriptions can participate directly in changing the space of reachable artifacts or states.

This is a weaker claim than “natural language has become code.” It is also more testable.

9. A discriminating research program

The theory should survive modification rather than merely name the current interface. Four experiments would place its central distinctions under pressure.

Experiment 1: semantic slack versus consequence coupling. Hold one intended transformation fixed and implement it as deterministic code, a declarative rule, a structured schema, a detailed prompt, and a minimal deictic prompt. Measure explicit information supplied, context dependence, run-to-run variance, correction required, and the degree of downstream state change. The hypothesis fails if the two proposed dimensions cannot be separated reliably.

Experiment 2: task location versus specification completion. Compare minimal labels, demonstrations, natural-language requirements, and iterative corrections for the same task. Test whether model behavior is best explained by stable activation of a learned capability or by incremental constraint accumulation.

Experiment 3: negative specification. Compare binary rejection, ranking, direct edit, gesture, and verbal critique. Measure whether a system converges toward stable human acceptance before the evaluator can state a positive criterion. The mechanism fails if later articulated criteria do not predict future judgments or if convergence depends entirely on explicit explanation.

Experiment 4: author-function across institutions. Give the same production graph and provenance record to evaluators working under dissertation, copyright, film-credit, and contributor-taxonomy rules. Observe which operations each regime treats as author-making. This would test whether the apparent authorship crisis is partly a mismatch among institutional attribution functions rather than a single metaphysical problem.

These experiments share a methodological commitment: compare mechanisms, not slogans. “Prompt,” “writing,” “program,” and “author” should remain provisional categories until the operations that make them useful have been identified.

10. Limits

Three limits are especially important.

First, semantic slack and consequence coupling are currently conceptual variables without validated measurement instruments. They may decompose into many dimensions. Context dependence, omission, ambiguity, variation tolerance, and reliance on model priors may not belong on one scale. Consequence may similarly require separate measures of immediacy, authority, reversibility, mediation, and scope.

Second, this account risks over-generalization from large language models to generative systems as a whole. Image models, multimodal agents, code generators, interactive worlds, and tool-using systems have different forms of context, action, and evaluation. The prompt-event must be instantiated rather than assumed to be uniform.

Third, the argument does not settle authorship. A production graph can make contribution

inspectable but cannot recover private intention or embodied judgment in full. Foucault’s author-function additionally warns that authorship is historically and institutionally variable. Provenance should therefore be treated as evidence about operations, not as a machine for automatically assigning credit.

11. Conclusion: from executable description to interpretive coupling

The most tempting theory of prompting says that description has become executable. That formulation is memorable and too smooth. Descriptions do not execute by themselves. Systems are built in which descriptions enter learned interpreters, acquire operational interpretations, and participate in changing reachable states.

The more precise object is the coupling.

Prompt-mediated writing becomes theoretically interesting when consequential action no longer requires the user to formalize every relevant distinction before computation begins. Learned priors, situated context, candidate realization, and recursive human judgment can carry part of the semantic burden. The specification is therefore neither wholly inside the initial prompt nor wholly inside the model. It develops across a sequence of interpretations and corrections.

This changes what programming can look like without making natural language identical to a programming language. It changes what writing can look like without reducing writing to delegation. And it changes the authorship problem without supplying a new sovereign author.

A useful research program follows the transformations rather than defending the categories. What did the expression leave unresolved? What did the interpreter supply? What changed in the reachable state space? What did failure make explicit? Which judgment altered the future of the artifact? Which institution later compressed that history into an author label?

Those questions move beyond the border between code and poetry. They describe a system in which interpretation itself has become part of the machinery connecting words to possible futures.

Appendix A — Assembly and provenance

This working paper was assembled from a bounded research checkpoint using a preserved source-and-zettel workflow. The public argument above does not treat the compilation machinery as evidence. Claim-level provenance is recorded separately in `interpretive-coupling__SOURCE_MAP.txt`. The exact assembly prompt is preserved at `_PROMPTS/004__SLIPCASE_v13.1__ASSEMBLY_PROMPT.txt`; its SHA-256 begins `8ac21ed6`. Earlier consequential prompts are preserved in `_PROMPTS/`. The portable reconstruction path is `000__RETURN_PATH.txt`; source inventory and bibliography are in `000__RESOURCES.txt` and `000__BIBLIOGRAPHY.txt`. The paper is a current wager, not evidence for its own claims.

References

- Austin, J. L. (1962). *How to Do Things with Words*. Ed. by J. O. Urmson. Oxford: Clarendon Press.
- Foucault, Michel (1977). “What Is an Author?” In: *Language, Counter-Memory, Practice: Selected Essays and Interviews*. Ed. by Donald F. Bouchard. Trans. by Donald F. Bouchard and Sherry Simon. Ithaca, NY: Cornell University Press, pp. 113–138.
- Geertz, Clifford (1973). “Thick Description: Toward an Interpretive Theory of Culture”. In: *The Interpretation of Cultures*. New York: Basic Books, pp. 3–30.
- Liu, Pengfei et al. (2023). “Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing”. In: *ACM Computing Surveys* 55.9, pp. 1–35. DOI: 10.1145/3560815.
- Polanyi, Michael (1966). *The Tacit Dimension*. Garden City, NY: Doubleday.
- Reynolds, Laria and Kyle McDonell (2021). “Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm”. In: *Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems*. DOI: 10.1145/3411763.3451760.
- Schön, Donald A. (1983). *The Reflective Practitioner: How Professionals Think in Action*. New York: Basic Books.
- Suchman, Lucy A. (1987). *Plans and Situated Actions: The Problem of Human-Machine Communication*. Cambridge: Cambridge University Press.